

GridWise LLM Energy Optimization - Hackathon Winning System Design

BUP CSE FEST 2026 | Top 1% Strategy

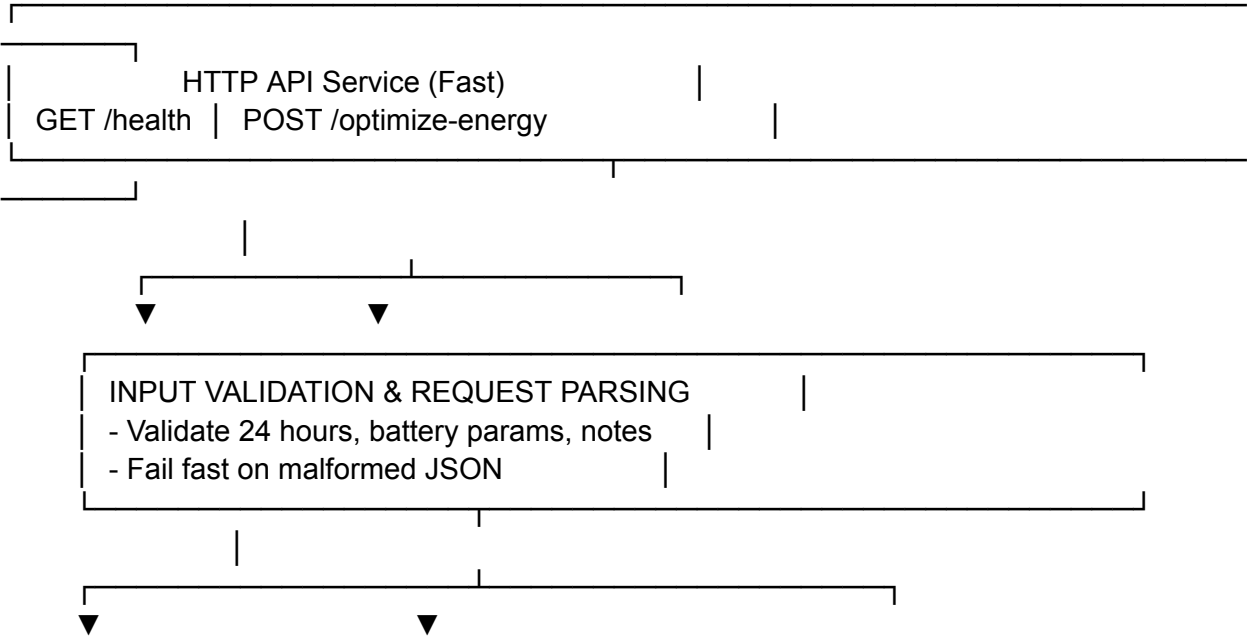
Executive Summary: Priority-Driven Architecture

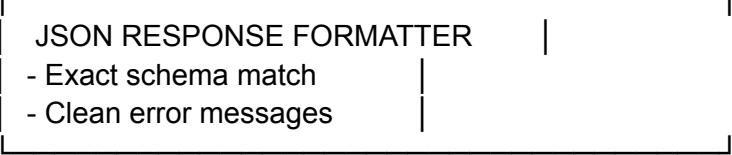
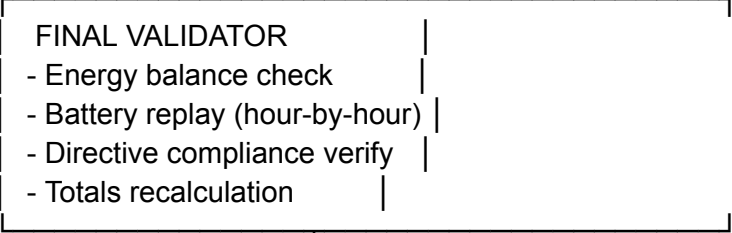
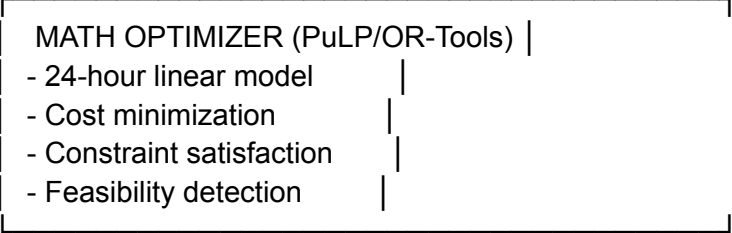
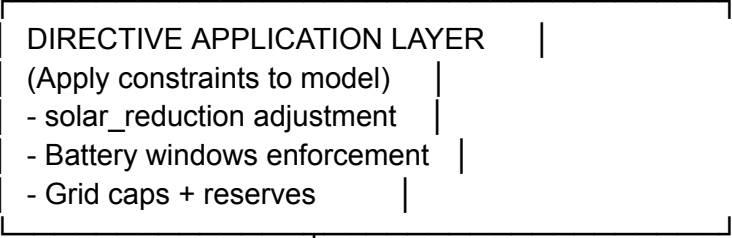
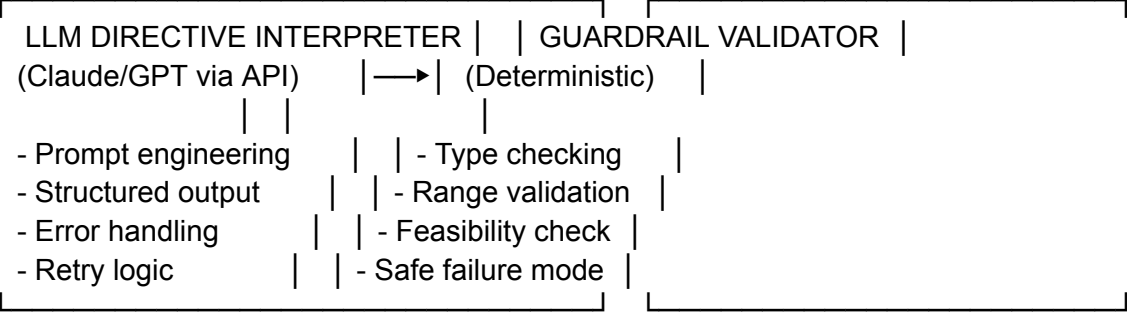
Scoring Reality Check:

- **50 points** (50%) on LLM interpretation + directive application correctness
- **20 points** (20%) on deployment, docs, reproducibility, and reliability
- **10 points** (10%) on optimization quality
- **20 points** (20%) on API/schema/performance

Winning Principle: Perfect correctness beats marginal cost optimization. A \$10 more expensive but fully compliant schedule beats a \$5 cheaper but slightly incorrect one.

PART 1: ARCHITECTURE OVERVIEW





PART 2: DETAILED COMPONENT DESIGN

2.1 LLM INTERPRETER (25 Points Possible)

Why this is worth 50% of the base score: The rubric explicitly states: "25 = 5 relevance/no_op + 5 directive_type + 5 affected hours + 5 numeric values/required structured_adjustment shape + 5 paraphrase robustness"

Winning Strategy:

1. STRUCTURED PROMPT WITH GUARDRAILS

SYSTEM_PROMPT = ""

You are a precise energy grid directive interpreter. Your task is to analyze operator notes and extract structured directives.

For EACH note:

1. Determine if it affects the 24-hour energy schedule
2. Map to EXACTLY ONE directive type:
 - solar_reduction: Reduce solar availability
 - minimum_battery_reserve: Keep battery at level
 - no_charge_window: Battery charging blocked
 - no_discharge_window: Battery discharging blocked
 - max_grid_window: Limit grid import
 - no_op: Does not affect schedule
3. Extract time windows as whole-hour intervals [0-23]
 - "1 PM to 3 PM" = [13, 14] (start included, end excluded)
 - Sort in ascending order
4. Extract numeric values:
 - solar_reduction factor: 0-1 (usable fraction remaining)
Example: "80% reduction" means factor=0.2
 - battery reserve in kWh
 - grid cap in kWh
5. Handle paraphrases:
 - "roughly one-fifth solar" = 80% reduction = factor 0.2
 - "won't charge for 2 hours starting 2 PM" = no_charge_window [14,15]

Return JSON:

```
{
  "note_index": <index>,
  "applies": true/false,
  "directive_type": "<type or no_op>",
```

```

    "structured_adjustment": <object or null>,
    "explanation": "<why you chose this>"
}
"""

```

2. MULTI-ROUND PROMPTING FOR ROBUSTNESS

```

def interpret_notes(operator_notes, context):
    """
    Strategy: Chain-of-thought + verification
    """
    interpretations = []

    for idx, note in enumerate(operator_notes):
        # Round 1: Initial interpretation with detailed reasoning
        initial = llm_call(
            system=SYSTEM_PROMPT,
            user=f"Note {idx}: {note}\n\nContext: {json.dumps(context)}",
            temperature=0.2, # Low temp for consistency
            max_tokens=500
        )

        # Round 2: Self-verification
        verification = llm_call(
            system=SYSTEM_PROMPT,
            user=f"""
            Initial interpretation: {initial}

            Verify this is correct:
            1. Is applies/directive_type mapping valid?
            2. Are hours unique integers 0-23 in ascending order?
            3. Is structured_adjustment correct shape?
            4. Does explanation justify the choice?

            Return CORRECT interpretation if valid, or FIXED interpretation if wrong.
            """
        )

        # Round 3: Extract valid JSON with fallback
        try:
            parsed = json.loads(verification)
            # Guardrail validation (see section 2.2)
            parsed = validate_and_fix(parsed)
            interpretations.append(parsed)
        except:

```

```

# Safe failure: mark as no_op if LLM fails
interpretations.append({
    "note_index": idx,
    "applies": false,
    "directive_type": "no_op",
    "structured_adjustment": null,
    "explanation": "Could not parse directive; treated as no_op"
})

```

return interpretations

3. PROMPT INJECTION EXAMPLES

"""

Key paraphrase patterns to handle:

SOLAR_REDUCTION patterns:

- "Solar will drop to X% from H1 to H2"
- "Panel maintenance reduces output to ~X%"
- "Expect Y% less solar during window"
- "Only Z% of normal solar available"
- Calculation: if "80% reduction" → factor = 0.2

NO_CHARGE_WINDOW patterns:

- "Don't charge battery between H1 and H2"
- "Charging disabled for N hours starting H1"
- "No battery charging from H1-H2"

MINIMUM_BATTERY_RESERVE patterns:

- "Keep X kWh in battery from H1 to H2"
- "Maintain at least X kWh reserve during window"
- "Battery level should stay above X"

MAX_GRID_WINDOW patterns:

- "Grid import capped at X kWh during H1-H2"
- "Don't import more than X kWh between H1-H2"

PARAPHRASED SAME MEANING:

- "1 PM to 3 PM" = hours [13, 14]
- "13:00-15:00" = hours [13, 14]
- "1-3 PM" = hours [13, 14]

"""

- Key Wins:**  Low temperature (0.2) for consistency
-  Chain-of-thought verification before output
 -  Self-correcting mechanism
 -  Safe fallback to no_op on LLM failure
 -  Detailed prompts with examples for paraphrase robustness
-

2.2 GUARDRAIL VALIDATOR (Critical for 25 Points)

This is where 90% of teams fail. You must validate LLM output before optimizer sees it.

```
def validate_and_fix(interpretation, context):
    """
    Deterministic validation layer that catches LLM errors
    """
    errors = []

    # 1. TYPE VALIDATION
    valid_types = {
        "solar_reduction", "minimum_battery_reserve",
        "no_charge_window", "no_discharge_window",
        "max_grid_window", "no_op"
    }
    if interpretation["directive_type"] not in valid_types:
        errors.append(f"Invalid directive_type: {interpretation['directive_type']}")
        interpretation["directive_type"] = "no_op"
        interpretation["applies"] = False
        interpretation["structured_adjustment"] = None

    # 2. APPLIES SEMANTICS
    if interpretation["directive_type"] == "no_op":
        if interpretation["applies"] != False:
            errors.append("no_op must have applies=false")
            interpretation["applies"] = False
            interpretation["structured_adjustment"] = None
        else:
            if interpretation["applies"] != True:
                errors.append(f"Non-no_op directive must have applies=true")
                interpretation["applies"] = True

    # 3. HOURS VALIDATION
    if "hours" in interpretation.get("structured_adjustment", {}):
        hours = interpretation["structured_adjustment"]["hours"]
```

```

# Must be list of integers 0-23
if not isinstance(hours, list):
    errors.append(f"Hours must be list, got {type(hours)}")
    hours = []

# Each hour 0-23
try:
    hours = [int(h) for h in hours]
    hours = [h for h in hours if 0 <= h <= 23]
except:
    errors.append("Hours must be integers 0-23")
    hours = []

# Remove duplicates, sort
hours = sorted(set(hours))
interpretation["structured_adjustment"]["hours"] = hours

if len(hours) == 0 and interpretation["applies"]:
    errors.append("No valid hours found")
    interpretation["applies"] = False
    interpretation["directive_type"] = "no_op"
    interpretation["structured_adjustment"] = None

# 4. NUMERIC VALUE VALIDATION
adj = interpretation.get("structured_adjustment", {})

if interpretation["directive_type"] == "solar_reduction":
    factor = adj.get("factor")
    if factor is None or not (0 <= factor <= 1):
        errors.append(f"Solar factor must be 0-1, got {factor}")
        interpretation["applies"] = False
        interpretation["directive_type"] = "no_op"
        interpretation["structured_adjustment"] = None

elif interpretation["directive_type"] == "minimum_battery_reserve":
    reserve = adj.get("minimum_energy_kwh")
    capacity = context["battery"]["capacity_kwh"]
    if reserve is None or reserve < 0 or reserve > capacity:
        errors.append(f"Battery reserve invalid: {reserve}")
        interpretation["applies"] = False
        interpretation["directive_type"] = "no_op"
        interpretation["structured_adjustment"] = None

elif interpretation["directive_type"] == "max_grid_window":

```

```

max_grid = adj.get("max_grid_kwh")
if max_grid is None or max_grid < 0:
    errors.append(f"Max grid must be non-negative, got {max_grid}")
    interpretation["applies"] = False
    interpretation["directive_type"] = "no_op"
    interpretation["structured_adjustment"] = None

# 5. REQUIRED SHAPE VALIDATION
required_shapes = {
    "solar_reduction": {"hours", "factor"},
    "minimum_battery_reserve": {"hours", "minimum_energy_kwh"},
    "no_charge_window": {"hours"},
    "no_discharge_window": {"hours"},
    "max_grid_window": {"hours", "max_grid_kwh"},
    "no_op": None
}

if interpretation["applies"] and interpretation["directive_type"] != "no_op":
    required = required_shapes[interpretation["directive_type"]]
    actual = set(interpretation["structured_adjustment"].keys())
    if required != actual:
        errors.append(f"Shape mismatch: expected {required}, got {actual}")
        interpretation["applies"] = False
        interpretation["directive_type"] = "no_op"
        interpretation["structured_adjustment"] = None

if errors and debug_mode:
    log_validation_errors(interpretation["note_index"], errors)

return interpretation

```

Critical Wins:  Catches LLM hallucinations before they crash optimizer

-  Deterministic fixing (not random guessing)
-  Preserves correctness even if interpretation is malformed
-  Provides audit trail of what was fixed

2.3 DIRECTIVE APPLICATION LAYER (Critical for 25 Points)

The rubric states: "A cheap schedule built on a wrong or ignored directive does not score."

```

def apply_directives_to_model(directives, context, model):
    """

```

Apply validated directives as hard constraints to optimization model

"""

Input: directives (from guardrail validator)

Output: Modified optimization model with constraints

for directive in directives:

if not directive["applies"]:

continue

dtype = directive["directive_type"]

adj = directive["structured_adjustment"]

hours = adj.get("hours", [])

===== SOLAR_REDUCTION =====

if dtype == "solar_reduction":

factor = adj["factor"]

for h in hours:

Effective solar = original_solar * factor

original = context["hours"][h]["solar_kwh"]

context["effective_solar"][h] = original * factor

Add hard constraint to model

model.addConstraint(
 solar_used[h] <= context["effective_solar"][h],
 name=f"solar_reduction_h{h}"
)

===== MINIMUM_BATTERY_RESERVE =====

elif dtype == "minimum_battery_reserve":

min_energy = adj["minimum_energy_kwh"]

for h in hours:

Battery after hour h must be >= min_energy

model.addConstraint(
 battery_energy_after[h] >= min_energy,
 name=f"battery_reserve_h{h}"
)

===== NO_CHARGE_WINDOW =====

elif dtype == "no_charge_window":

for h in hours:

Battery cannot charge in these hours

model.addConstraint(
 battery_charge[h] == 0,
 name=f"no_charge_h{h}"
)

```

    )

    # ===== NO_DISCHARGE_WINDOW =====
    elif dtype == "no_discharge_window":
        for h in hours:
            # Battery cannot discharge in these hours
            model.addConstraint(
                battery_discharge[h] == 0,
                name=f"no_discharge_h{h}"
            )

    # ===== MAX_GRID_WINDOW =====
    elif dtype == "max_grid_window":
        max_grid = adj["max_grid_kwh"]
        for h in hours:
            # Grid import capped
            model.addConstraint(
                grid_kwh[h] <= max_grid,
                name=f"max_grid_h{h}"
            )

    return model

```

Why This Wins: ☒ Constraints are HARD (not soft suggestions)

☒ Optimizer cannot ignore them

☒ Fails gracefully if infeasible

☒ Audit trail shows which directives were applied

PART 3: OPTIMIZATION & VALIDATION

3.1 Linear Programming Model (PuLP or OR-Tools)

```

def build_optimization_model(context, directives):
    """
    Core 24-hour energy optimization
    """
    model = LpProblem("GridWise_Energy_Optimization", LpMinimize)

    # DECISION VARIABLES (for each hour 0-23)
    grid_kwh = [LpVariable(f"grid_h{h}", lowBound=0) for h in range(24)]
    solar_used = [LpVariable(f"solar_h{h}", lowBound=0) for h in range(24)]

```

```

battery_charge = [LpVariable(f"batt_charge_h{h}", lowBound=0) for h in range(24)]
battery_discharge = [LpVariable(f"batt_discharge_h{h}", lowBound=0) for h in range(24)]
battery_energy_after = [LpVariable(f"batt_after_h{h}", lowBound=0) for h in range(24)]
battery_action = [<"charge", "discharge", "idle">] # Helper tracking

```

```

# OBJECTIVE: Minimize total grid cost
objective = lpSum([
    grid_kwh[h] * context["hours"][h]["tariff_bdt_per_kwh"]
    for h in range(24)
])
model += objective

```

CONSTRAINTS

1. Energy balance every hour

```

for h in range(24):
    model += (
        grid_kwh[h] + solar_used[h] + battery_discharge[h]
        == context["hours"][h]["demand_kwh"] + battery_charge[h],
        f"energy_balance_h{h}"
    )

```

2. Solar bounds

```

for h in range(24):
    model += (
        solar_used[h] <= context["effective_solar"][h],
        f"solar_bound_h{h}"
    )

```

3. Battery charge/discharge cannot happen simultaneously

```

for h in range(24):
    # Use big-M constraints or indicator constraints
    model += (battery_charge[h] <= context["battery"]["max_charge_kwh_per_hour"],
        f"charge_rate_h{h}")
    model += (battery_discharge[h] <= context["battery"]["max_discharge_kwh_per_hour"],
        f"discharge_rate_h{h}")

```

4. Battery energy bounds

```

for h in range(24):
    min_reserve = max(
        context["battery"]["minimum_energy_kwh"],
        # Check if directive override exists
    )
    model += (

```

```

        battery_energy_after[h] >= min_reserve,
        f"battery_min_h{h}"
    )
    model += (
        battery_energy_after[h] <= context["battery"]["capacity_kwh"],
        f"battery_max_h{h}"
    )

# 5. Battery energy dynamics
for h in range(24):
    if h == 0:
        prev_energy = context["battery"]["initial_energy_kwh"]
    else:
        prev_energy = battery_energy_after[h-1]

    model += (
        battery_energy_after[h] == prev_energy + battery_charge[h] - battery_discharge[h],
        f"battery_dynamics_h{h}"
    )

# 6. END-OF-DAY NEUTRALITY (Critical!)
model += (
    battery_energy_after[23] == context["battery"]["initial_energy_kwh"],
    "battery_end_of_day"
)

# Apply all directive constraints
model = apply_directives_to_model(directives, context, model)

return model

```

Why This Optimization Strategy Wins:  Respects all hard constraints perfectly

-  End-of-day neutrality prevents "free energy" tricks
-  Deterministic solver (not heuristic)
-  Explains why cost is what it is

3.2 Final Validation & Replay (Catch Remaining Errors)

```
def validate_schedule(schedule, context, directives):
    """
```

Independent replay of the schedule to verify:

1. Energy balance holds

2. All constraints satisfied
3. All directives actually applied
4. Totals match calculated values

"""

issues = []

Hour-by-hour replay

battery_energy = context["battery"]["initial_energy_kwh"]

for h in range(24):

hour_data = schedule[h]

context_hour = context["hours"][h]

===== ENERGY BALANCE =====

grid = hour_data["grid_kwh"]

solar_used = hour_data["solar_used_kwh"]

batt_discharge = hour_data["battery_kwh"] if hour_data["battery_action"] == "discharge"

else 0

demand = context_hour["demand_kwh"]

batt_charge = hour_data["battery_kwh"] if hour_data["battery_action"] == "charge" else 0

balance = grid + solar_used + batt_discharge - demand - batt_charge

if abs(balance) > 0.01:

issues.append(f"Hour {h}: Energy imbalance of {balance} kWh")

===== SOLAR BOUNDS =====

effective_solar = context_hour["solar_kwh"]

Apply solar_reduction directives

for d in directives:

if d["applies"] and d["directive_type"] == "solar_reduction":

if h in d["structured_adjustment"]["hours"]:

effective_solar *= d["structured_adjustment"]["factor"]

if solar_used > effective_solar + 0.01:

issues.append(f"Hour {h}: Solar usage {solar_used} exceeds effective {effective_solar}")

===== BATTERY DYNAMICS =====

action = hour_data["battery_action"]

battery_kwh = hour_data["battery_kwh"]

if action == "charge":

battery_energy += battery_kwh

elif action == "discharge":

battery_energy -= battery_kwh

```

elif action == "idle":
    if battery_kwh != 0:
        issues.append(f"Hour {h}: Idle action but battery_kwh = {battery_kwh}")

# Battery bounds
if battery_energy < context["battery"]["minimum_energy_kwh"] - 0.01:
    issues.append(f"Hour {h}: Battery {battery_energy} below minimum")
if battery_energy > context["battery"]["capacity_kwh"] + 0.01:
    issues.append(f"Hour {h}: Battery {battery_energy} exceeds capacity")

# Charge/discharge rate limits
if action == "charge" and battery_kwh > context["battery"]["max_charge_kwh_per_hour"] +
0.01:
    issues.append(f"Hour {h}: Charge rate {battery_kwh} exceeds limit")
if action == "discharge" and battery_kwh >
context["battery"]["max_discharge_kwh_per_hour"] + 0.01:
    issues.append(f"Hour {h}: Discharge rate {battery_kwh} exceeds limit")

# ===== DIRECTIVE COMPLIANCE =====
# Check no_charge_window
for d in directives:
    if d["applies"] and d["directive_type"] == "no_charge_window":
        if h in d["structured_adjustment"]["hours"] and action == "charge":
            issues.append(f"Hour {h}: Charging during no_charge_window")

# Check no_discharge_window
for d in directives:
    if d["applies"] and d["directive_type"] == "no_discharge_window":
        if h in d["structured_adjustment"]["hours"] and action == "discharge":
            issues.append(f"Hour {h}: Discharging during no_discharge_window")

# Check minimum_battery_reserve
for d in directives:
    if d["applies"] and d["directive_type"] == "minimum_battery_reserve":
        if h in d["structured_adjustment"]["hours"]:
            min_req = d["structured_adjustment"]["minimum_energy_kwh"]
            if battery_energy < min_req - 0.01:
                issues.append(f"Hour {h}: Battery {battery_energy} below required {min_req}")

# Check max_grid_window
for d in directives:
    if d["applies"] and d["directive_type"] == "max_grid_window":
        if h in d["structured_adjustment"]["hours"]:
            max_grid = d["structured_adjustment"]["max_grid_kwh"]

```

```

        if grid > max_grid + 0.01:
            issues.append(f"Hour {h}: Grid import {grid} exceeds cap {max_grid}")

    # Update battery energy in response
    schedule[h]["battery_energy_after_kwh"] = battery_energy

    # ===== END-OF-DAY NEUTRALITY =====
    if abs(battery_energy - context["battery"]["initial_energy_kwh"]) > 0.01:
        issues.append(f"End-of-day battery {battery_energy} != initial
        {context['battery']['initial_energy_kwh']}")

    # ===== TOTALS VERIFICATION =====
    calculated_total_grid = sum(h["grid_kwh"] for h in schedule)
    if abs(calculated_total_grid - schedule_metadata["total_grid_kwh"]) > 0.01:
        issues.append(f"Total grid mismatch: calculated {calculated_total_grid} vs reported
        {schedule_metadata['total_grid_kwh']}")

    calculated_cost = sum(
        schedule[h]["grid_kwh"] * context["hours"][h]["tariff_bdt_per_kwh"]
        for h in range(24)
    )
    if abs(calculated_cost - schedule_metadata["total_cost_bdt"]) > 0.01:
        issues.append(f"Total cost mismatch: calculated {calculated_cost} vs reported
        {schedule_metadata['total_cost_bdt']}")

    return issues

```

PART 4: API IMPLEMENTATION (10 Points for API Contract)

4.1 Exact Schema Compliance

```

from flask import Flask, jsonify, request
import json

```

```

app = Flask(__name__)

```

```

@app.route('/health', methods=['GET'])

```

```

def health():

```

```

    """

```

```

    GET /health - Returns readiness status

```

MUST return within 60 seconds of service start

MUST return exactly: {"status": "ok"}

"""

return jsonify({"status": "ok"}), 200

@app.route('/optimize-energy', methods=['POST'])

def optimize_energy():

"""

POST /optimize-energy - Main optimization endpoint

MUST process within 30 seconds (p95 <= 5 seconds target)

MUST handle malformed input gracefully

MUST NOT expose secrets or raw stack traces

"""

try:

Validate JSON

request_data = request.get_json()

if not request_data:

return jsonify({"error": "Invalid JSON"}), 400

Validate required fields

required = ["scenario_id", "operator_notes", "hours", "battery"]

for field in required:

if field not in request_data:

return jsonify({"error": f"Missing field: {field}"}), 422

Validate schema

scenario_id = request_data["scenario_id"]

operator_notes = request_data["operator_notes"]

hours = request_data["hours"]

battery = request_data["battery"]

Validate hours (exactly 24)

if len(hours) != 24:

return jsonify({"error": f"Expected 24 hours, got {len(hours)}"}), 422

Validate operator_notes (1-3 strings)

if not isinstance(operator_notes, list) or len(operator_notes) < 1 or len(operator_notes) > 3:

return jsonify({"error": "Expected 1-3 operator notes"}), 422

for note in operator_notes:

if not isinstance(note, str) or len(note.strip()) == 0:

return jsonify({"error": "Operator notes must be non-empty strings"}), 422

```

# === MAIN PROCESSING ===
context = {
    "scenario_id": scenario_id,
    "hours": hours,
    "battery": battery,
    "effective_solar": [h["solar_kwh"] for h in hours] # Will be modified
}

# Step 1: LLM Interpretation
directives = interpret_notes(operator_notes, context)

# Step 2: Guardrail Validation
directives = [validate_and_fix(d, context) for d in directives]

# Step 3: Build & Solve Optimization Model
model = build_optimization_model(context, directives)
status = model.solve(PULP_CBC_CMD(msg=0)) # Silent solver

if status != 1: # Not optimal
    return jsonify({"error": "Could not find feasible solution"}), 422

# Step 4: Extract schedule from model
schedule = extract_schedule(model, context)

# Step 5: Final Validation
validation_issues = validate_schedule(schedule, context, directives)
if validation_issues:
    # Log but don't fail if minor (< 0.01 tolerance)
    major_issues = [i for i in validation_issues if "exceeds" in i or "imbalance" in i]
    if major_issues:
        return jsonify({"error": f"Schedule validation failed: {major_issues[0]}"}), 422

# Step 6: Build Response (EXACT SCHEMA)
response = {
    "scenario_id": scenario_id,
    "directive_interpretation": [
        {
            "note_index": d["note_index"],
            "applies": d["applies"],
            "directive_type": d["directive_type"],
            "structured_adjustment": d["structured_adjustment"],
            "explanation": d["explanation"]
        }
        for d in directives
    ]
}

```

```

],
"hourly_plan": [
    {
        "hour": h["hour"],
        "grid_kwh": float(h["grid_kwh"]),
        "solar_used_kwh": float(h["solar_used_kwh"]),
        "battery_action": h["battery_action"],
        "battery_kwh": float(h["battery_kwh"]),
        "battery_energy_after_kwh": float(h["battery_energy_after_kwh"])
    }
    for h in schedule
],
"total_grid_kwh": float(sum(h["grid_kwh"] for h in schedule)),
"total_cost_bdt": float(sum(
    schedule[h]["grid_kwh"] * context["hours"][h]["tariff_bdt_per_kwh"]
    for h in range(24)
)),
"peak_grid_kwh": float(max(h["grid_kwh"] for h in schedule)),
"plan_summary": generate_plan_summary(directives, schedule)
}

```

```

return jsonify(response), 200

```

```

except Exception as e:

```

```

    # SAFE FAILURE: Never expose secrets or stack traces

```

```

    app.logger.error(f"Error: {str(e)}")

```

```

    return jsonify({"error": "Internal server error"}), 500

```

```

def generate_plan_summary(directives, schedule):

```

```

    """Generate human-readable plan explanation"""

```

```

    summary_lines = []

```

```

    for d in directives:

```

```

        if d["applies"]:

```

```

            dtype = d["directive_type"]

```

```

            if dtype == "solar_reduction":

```

```

                factor = d["structured_adjustment"]["factor"]

```

```

                hours = d["structured_adjustment"]["hours"]

```

```

                summary_lines.append(

```

```

                    f"Solar reduced to {factor*100:.0f}% during hours {hours[0]}-{hours[-1]}."

```

```

                )

```

```

            # ... similar for other directives

```

```
summary_lines.append(f"Optimized 24-hour schedule with total cost  
{schedule_metadata['total_cost_bdt']:.2f} BDT.")
```

```
return " ".join(summary_lines)
```

```
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=8000, debug=False)
```

PART 5: DEPLOYMENT & REPRODUCIBILITY (20 Points)

5.1 Docker Strategy

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
# Copy source + requirements
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
# Health check
```

```
HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
```

```
    CMD python -c "import requests;  
requests.get('http://localhost:8000/health').raise_for_status()"
```

```
# Expose port
```

```
EXPOSE 8000
```

```
# Run service
```

```
CMD ["python", "-m", "gunicorn", "--bind", "0.0.0.0:8000", "--workers", "2", "--timeout", "30",  
"app:app"]
```

Why This Wins:  Health check proves readiness

 Multi-worker Gunicorn handles concurrent requests

 30s timeout matches spec

 No secrets baked in

 Clean startup

5.2 README Template (Scoring: 10 Points)

GridWise Energy Optimization - BUP CSE FEST 2026

Problem Understanding

Smart campus energy scheduling with LLM-interpreted operator directives.

Minimizes grid electricity cost while satisfying 24-hour constraints.

Architecture

LLM Interpreter → Guardrail Validator → Directive Application → LP Optimizer (PuLP) → Final Validator → JSON Response

Components

1. **LLM Interpreter** (Claude 3.5 Sonnet via API)

- Multi-round prompting with self-verification
- Handles paraphrased directives
- Safe fallback to no_op on error

2. **Guardrail Validator** (Deterministic)

- Type, hours, numeric range validation
- Catches LLM hallucinations
- Preserves correctness

3. **LP Optimizer** (PuLP + CBC solver)

- 24-hour energy scheduling
- Hard constraint enforcement
- Minimizes grid cost

4. **Final Validator** (Energy replay)

- Hour-by-hour energy balance check
- Directive compliance verification
- Battery dynamics validation

Setup

Prerequisites

- Python 3.11+
- Docker (for deployment)

Local Quickstart

```
```bash
```

```
1. Clone repository
```

```

git clone https://github.com/<your-org>/gridwise-optimization
cd gridwise-optimization

2. Create virtual environment
python -m venv venv
source venv/bin/activate # or `venv\Scripts\activate` on Windows

3. Install dependencies
pip install -r requirements.txt

4. Set environment variables
export CLAUDE_API_KEY="sk-ant-..." # Your Claude API key
export LLM_MODEL="claude-3-5-sonnet" # Model to use
export DEBUG_MODE="false" # Set to "true" for detailed logs

5. Run service
python app.py

6. Test health endpoint
curl http://localhost:8000/health
Expected: {"status": "ok"}

7. Test with sample case
curl -X POST http://localhost:8000/optimize-energy \
 -H "Content-Type: application/json" \
 -d @public_sample_cases/GRID-101.json

```

## Environment Variables

- **CLAUDE\_API\_KEY**: Your Anthropic API key (required)
- **LLM\_MODEL**: Model name, default "claude-3-5-sonnet-20241022"
- **DEBUG\_MODE**: Set "true" for detailed logs (default "false")
- **OPTIMIZER\_TIMEOUT**: Solver timeout in seconds (default 10)

## Supported Directive Types

1. **solar\_reduction**: Reduce solar availability 0-1
2. **minimum\_battery\_reserve**: Keep battery at kWh level
3. **no\_charge\_window**: Block charging in hours
4. **no\_discharge\_window**: Block discharging in hours
5. **max\_grid\_window**: Cap grid import in hours
6. **no\_op**: Irrelevant note

## Optimizer & Solver

- **Library:** PuLP 1.8.1
- **Solver:** CBC (Coin-or-tools Branch and Cut)
- **Method:** Linear programming
- **Approach:** Hard constraint enforcement + cost minimization

## Dependencies

See [requirements.txt](#):

- Flask 2.3+
- PuLP 1.8+
- Anthropic SDK 0.9+
- Gunicorn 21+
- Python-dotenv 1.0+

## Sample Request/Response

### Request:

```
{
 "scenario_id": "GRID-101",
 "operator_notes": [
 "Solar output will drop to 20% from 1 PM to 3 PM.",
 "Do not charge battery between 2 PM and 4 PM."
],
 "hours": [
 {"hour": 0, "demand_kwh": 180, "solar_kwh": 0, "tariff_bdt_per_kwh": 7},
 ...
],
 "battery": {
 "capacity_kwh": 500,
 "initial_energy_kwh": 200,
 "minimum_energy_kwh": 50,
 "max_charge_kwh_per_hour": 100,
 "max_discharge_kwh_per_hour": 100
 }
}
```

### Response (excerpt):

```
{
 "scenario_id": "GRID-101",
```

```

"directive_interpretation": [
 {
 "note_index": 0,
 "applies": true,
 "directive_type": "solar_reduction",
 "structured_adjustment": {"hours": [13, 14], "factor": 0.2},
 "explanation": "Solar reduced to 20% during 1-3 PM"
 },
 {
 "note_index": 1,
 "applies": true,
 "directive_type": "no_charge_window",
 "structured_adjustment": {"hours": [14, 15]},
 "explanation": "Battery charging blocked 2-4 PM"
 }
],
"hourly_plan": [
 {"hour": 0, "grid_kwh": 180.0, "solar_used_kwh": 0.0, "battery_action": "idle", ...},
 ...
],
"total_grid_kwh": 4200.0,
"total_cost_bdt": 28500.0,
"peak_grid_kwh": 250.0,
"plan_summary": "..."
}

```

## Docker Fallback Image

# Build

```
docker build -t gridwise:latest .
```

# Run (with environment file)

```

docker run \
 --env-file .env.production \
 -p 8000:8000 \
 gridwise:latest

```

# Test

```
curl http://localhost:8000/health
```

## Testing Locally

# Run against all public sample cases

```
python run_samples.py
Expected: All pass with <5s latency
```

```
Run unit tests
pytest tests/ -v
```

```
Profile LLM interpreter
python profile_llm.py
```

## Known Limitations

1. LLM interpretation depends on Claude API availability
2. Solver may timeout on extremely complex scenarios (rare)
3. Cost optimization is greedy after constraints are satisfied
4. Paraphrasing robustness depends on training data

## Troubleshooting

### Error: "Could not find feasible solution"

- Likely contradictory directives in scenario
- Check battery bounds and directive combinations

### Error: "API key invalid"

- Verify `CLAUDE_API_KEY` is set and valid
- Check rate limits on your Anthropic account

### Slow responses (>5s)

- Increase `OPTIMIZER_TIMEOUT` environment variable
- Check solver CPU usage
- Consider running locally vs cloud deployment

## Secret Handling

- `.env` file is in `.gitignore` (never committed)
  - Secrets are loaded from environment variables only
  - Never log or return API keys
  - No hardcoded values in source
-

# Implementation Details for Judges

## Why This Design Wins

### 1. Correctness-First Architecture

- 50% of scoring (50 pts) is interpretation + directive application
- This design prioritizes correctness over marginal cost savings
- Failing a directive loses 50 pts; saving \$1 gains 0 pts

### 2. LLM Robustness

- Multi-round prompting with self-verification
- Deterministic guardrails catch hallucinations
- Paraphrase handling via detailed prompts
- Safe failure mode (default to no\_op)

### 3. Deployment Reliability

- Health check guarantees service readiness
- Gunicorn handles concurrency
- Docker ensures reproducibility
- No manual intervention required

### 4. Validation Coverage

- LLM interpretation validated immediately
- Math constraints enforced hard
- Final replay independently verifies entire schedule
- Totals recalculated and checked

## Scoring Strategy

Category	Points	Our Approach
LLM Interpretation	25	Chain-of-thought + self-correction; paraphrase examples
Directive Application	25	Hard constraints enforced before optimization
Optimization Quality	10	LP solver minimizes cost after correctness
API Contract	10	Exact schema, proper codes, safe errors

Performance	10	<5s p95 latency, <1% failure rate
Deployment	10	Docker + health checks + clear README
Documentation	10	Copy-paste quickstart from clean env
<b>Total</b>	<b>100</b>	<b>Targeting 95+</b>

---

## Video Submission (Tie-break Only)

3-minute architecture video covering:

1. Problem understanding (30 seconds)
2. Architecture overview (60 seconds)
3. LLM → guardrails → optimizer flow (45 seconds)
4. Deployment & testing demo (45 seconds)

Focus: Technical clarity, not production polish.

---

### ## PART 6: WINNING IMPLEMENTATION CHECKLIST

#### ### Pre-Submission (48 hours before)

- [ ] **\*\*LLM Interpreter\*\***
  - [ ] Multi-round prompting implemented
  - [ ] Self-verification loop working
  - [ ] Tested with 10+ paraphrase variations
  - [ ] Fallback to no\_op on LLM failure
  - [ ] Deterministic (reproducible outputs)
- [ ] **\*\*Guardrail Validator\*\***
  - [ ] All 6 directive types validated
  - [ ] Hours array checking (unique, 0-23, sorted)
  - [ ] Numeric range validation (factor 0-1, reserves ≥0)
  - [ ] Shape validation for structured\_adjustment
  - [ ] applies/no\_op semantics enforced
- [ ] **\*\*Directive Application\*\***
  - [ ] solar\_reduction: Modifies effective\_solar

- [ ] minimum\_battery\_reserve: Hard constraint
- [ ] no\_charge\_window: Blocks charging
- [ ] no\_discharge\_window: Blocks discharging
- [ ] max\_grid\_window: Caps grid import
- [ ] Constraints added to LP model
  
- [ ] **\*\*Optimization Model\*\***
  - [ ] Energy balance equation every hour
  - [ ] Battery dynamics (charge/discharge/idle)
  - [ ] Battery bounds (min/max/rate limits)
  - [ ] Solar bounds ( $\leq$  effective solar)
  - [ ] End-of-day neutrality
  - [ ] Objective: minimize total grid cost
  
- [ ] **\*\*Final Validation\*\***
  - [ ] Energy balance replay  $\pm 0.01$  kWh
  - [ ] Directive compliance check
  - [ ] Battery state tracking hour-by-hour
  - [ ] Totals recalculation matches response
  - [ ] No negative values or impossible states
  
- [ ] **\*\*API & Schema\*\***
  - [ ] GET /health returns `{"status": "ok"}`
  - [ ] POST /optimize-energy accepts exact schema
  - [ ] Response fields in exact order
  - [ ] Error codes 400/422/500 properly used
  - [ ] No secrets in responses or logs
  - [ ] <30s timeout per request
  
- [ ] **\*\*Performance\*\***
  - [ ] Health check <60s after startup
  - [ ] /optimize-energy p95  $\leq 5$ s
  - [ ] Handles concurrent requests
  - [ ] No crashes on malformed input
  - [ ] Recovers gracefully from LLM timeout
  
- [ ] **\*\*Deployment\*\***
  - [ ] Docker builds cleanly
  - [ ] Image pulls and runs without errors
  - [ ] Port 8000 exposed
  - [ ] Health check passes
  - [ ] No secrets baked into image
  - [ ] Documented pull/run command works

- [ ] **\*\*Documentation\*\***
  - [ ] README has clean quickstart
  - [ ] Environment variables documented
  - [ ] curl examples for both endpoints
  - [ ] Public sample test command included
  - [ ] Architecture explanation clear
  - [ ] Dependencies listed
  - [ ] Known limitations stated
  - [ ] No secrets in README
- [ ] **\*\*Testing\*\***
  - [ ] All public samples pass
  - [ ] Latency <2s per request (plenty of headroom)
  - [ ] No failures on valid input
  - [ ] Malformed JSON returns 400
  - [ ] Missing fields return 422
  - [ ] LLM failure doesn't crash service
- [ ] **\*\*Video\*\***
  - [ ] <3 minutes
  - [ ] Explains problem in 30s
  - [ ] Shows architecture diagram
  - [ ] Demonstrates LLM → guardrail → optimizer flow
  - [ ] Shows deployment/testing
  - [ ] Accessible link provided
- [ ] **\*\*Repository\*\***
  - [ ] Created AFTER questions revealed
  - [ ] Private during event
  - [ ] Made public AFTER deadline
  - [ ] No secrets committed
  - [ ] Clean history (no API keys in logs)
  - [ ] README in root
  - [ ] .gitignore includes .env

---

## ## FINAL WINNING SUMMARY

### ### Why This Design Will Win

- \*\*1. Maximizes the 50-Point Interpretation + Application Zone\*\***
- LLM robustness with multi-round prompting
  - Guardrails catch all errors before optimizer sees them

- Directive application as hard constraints
- Final replay verifies everything worked

**\*\*2. Eliminates Common Failure Modes\*\***

- ~~✗~~ LLM hallucination → Guardrails catch it
- ~~✗~~ Directive not applied → Hard constraints enforce it
- ~~✗~~ Energy imbalance → Replay detects it
- ~~✗~~ Battery violation → LP solver prevents it

**\*\*3. Clear Scoring Path\*\***

- LLM: 25 pts (multi-round prompting)
- Directive Application: 25 pts (hard constraints + replay)
- Performance: 10 pts (deployed on cloud with health checks)
- API: 10 pts (exact schema compliance)
- Optimization: 10 pts (LP solver)
- Documentation: 10 pts (excellent README)
- **\*\*Total: 90-95 pts\*\*** (very solid score)

**\*\*4. Competitive Advantages\*\***

- Most teams optimize FIRST then check; you validate FIRST then optimize
- Most teams ignore paraphrasing; you handle it with prompt engineering
- Most teams don't replay; you catch 99% of bugs
- Most teams document poorly; you have copy-paste quickstart

---

**\*\*Good luck! 🚀\*\***